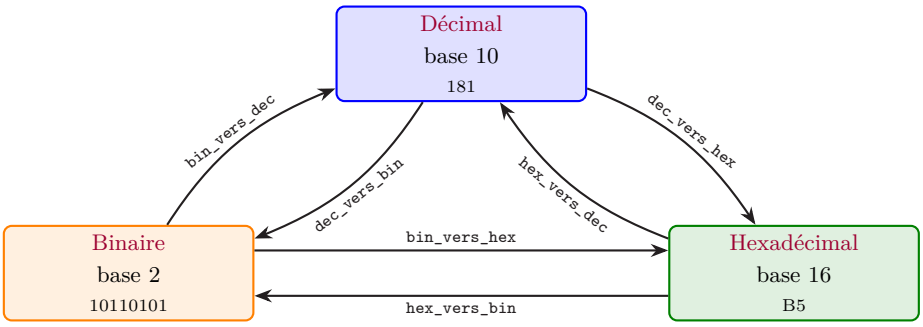
Bases de numération

Les types de base

Cette fiche t'accompagne pour programmer les conversions entre les bases 2, 10 et 16. La théorie a déjà été vue en cours : ici, on revient à l'essentiel et on te guide pas à pas vers le code. Les exercices marqués ★ sont des échauffements, ★★ demandent de la réflexion, ★★★ sont des défis. Dans les codes, les _____ sont à compléter.

1. Vue d'ensemble et règles du jeu

Tout au long de la fiche, un nombre écrit dans une base est représenté par une chaîne de caractères (`"10110101"`, `"B5"`) et un nombre « pur » par un entier Python (`181`). Nous écrirons six fonctions qui forment le réseau suivant :



RAPPEL – valeur d'un nombre écrit en base b

Si un nombre s'écrit $d_{n-1}\dots d_1 d_0$ en base b (chaque d_i est un chiffre entre 0 et $b-1$), sa valeur est

$$d_{n-1}\,b^{n-1} + \cdots + d_1\,b + d_0 = \sum_{i=0}^{n-1} d_i\,b^i.$$

Notre exemple fil rouge : $(10110101)_2 = 128+32+16+4+1 = (181)_{10} = (B5)_{16}$ car $11\times 16+5 = 181$.

RÈGLES DU JEU

Tu n'as pas le droit d'utiliser `bin()`, `hex()`, `int(s, 2)` ou les formats `{:b}`, `{:x}`. Tu peux en revanche les utiliser après coup, dans la console, pour vérifier tes résultats : `bin(181)` donne `'0b10110101'` et `hex(181)` donne `'0xb5'`.

2. Boîte à outils Python sur les chaînes

Rappels (console Python)

```
s = "10110101"
len(s)              # 8         : nombre de caractères
s[0]                # "1"       : premier caractère (indice 0)
s[-1]               # "1"       : dernier caractère
s[2:6]              # "1101"    : caractères d'indices 2, 3, 4, 5
s[::-1]             # "10101101" : chaîne retournée
"0" + s             # "010110101" : concaténation (ajout à gauche)

for c in s:         # c vaut successivement "1", "0", "1", "1", ...
    print(c)

int("7")            # 7         : caractère -> entier (un chiffre décimal)
str(7)              # "7"       : entier -> caractère
CHIFFRES = "0123456789ABCDEF"
CHIFFRES[11]        # "B"       : position -> chiffre
CHIFFRES.index("B")   # 11      : chiffre -> position

17 // 5             # 3         : quotient de la division euclidienne
17 % 5              # 2         : reste de la division euclidienne
```

NE PAS CONFONDRE

`"7"` est un caractère (type `str`), `7` est un entier (type `int`). `"7" + "1"` donne `"71"` alors que `7 + 1` donne `8`. De même, `/` donne un flottant : pour une division euclidienne, utilise toujours `//` et `%`.

Exercice 1 Reconnaître un nombre binaire ★

Complète la fonction qui renvoie `True` si la chaîne ne contient que des `"0"` et des `"1"`.

```
def est_binaire(s):
    for c in s:
        if c ____________________:
            return False
    return _______
```

Exercice 2 Retourner une chaîne sans les tranches ★★

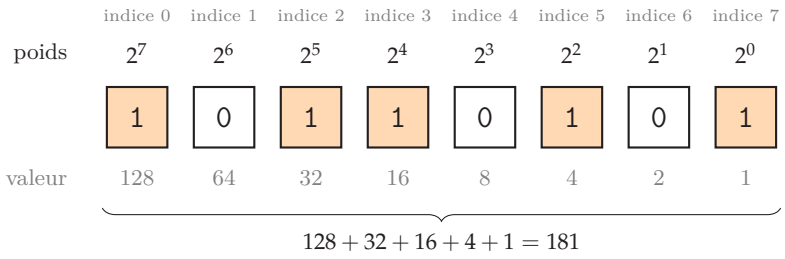
Complète cette fonction qui renvoie la chaîne `s` à l'envers sans utiliser `[::-1]`. Quel est le rôle de l'ordre dans l'expression à droite du signe `=` ? Garde bien cette astuce en tête : elle va resservir.

```
def inverser(s):
    r = ""
    for c in s:
        r = ________________
    return r
```

3. Du binaire au décimal

A. Idée 1 : les poids

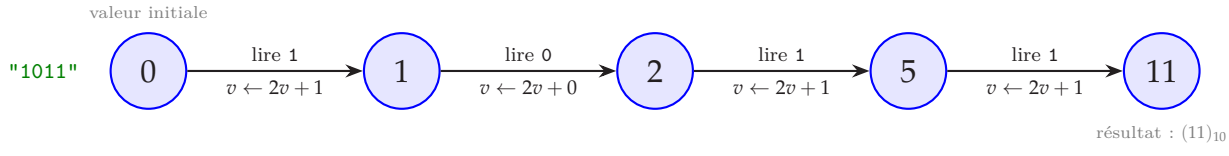
Chaque bit a un poids égal à une puissance de 2 : on additionne les poids des bits qui valent 1.



Remarque : Le bit d'indice 0 dans la chaîne (le plus à gauche) est celui de poids le plus fort. Dans une chaîne `s` de longueur n, le caractère `s[i]` a pour poids 2^{n-1-i}.

B. Idée 2 : la méthode de Horner (la plus pratique en code)

On lit la chaîne de gauche à droite en gardant une valeur v (au départ $v = 0$). À chaque chiffre lu, on décale ce qu'on a déjà d'un rang en multipliant par la base, puis on ajoute le nouveau chiffre : $v \leftarrow 2v + \text{chiffre}$.



Exercice 3 À la main ★

Convertis en décimal (utilise la méthode de ton choix).

`1101`	→	_____		`100000`	→	_____
`11111111`	→	_____		`10101010`	→	_____

Exercice 4 Dérouler l'algorithme ★

Complète le tableau d'exécution de la fonction ci-dessous pour `bin_vers_dec("1101")`, puis complète le code.

caractère `c`	`valeur` avant	`valeur` après
`"1"`	0	_____
`"1"`	_____	_____
`"0"`	_____	_____
`"1"`	_____	_____

```
def bin_vers_dec(s):
    valeur = ______
    for c in s:
        valeur = valeur * ______ + ____________
    return valeur
```

Exercice 5 Variante avec les poids ★★

Complète cette seconde version, qui utilise l'indice et la puissance de 2 de chaque bit. Compare : laquelle te semble la plus simple ? la plus rapide ?

```
def bin_vers_dec_v2(s):
    n = len(s)
    total = 0
    for i in range(____________):
        # s[i] a pour poids 2 puissance (n - 1 - i)
        total = total + int(s[i]) * 2 ** _______________
    return total
```

Exercice 6 Tester, c'est douter ★

Une fonction n'est correcte que si elle est testée. Place ces tests sous ta fonction, puis ajoute-en trois de ton choix (pense aux cas limites : `"0"`, `"1"`, une chaîne avec des zéros au début).

```
assert bin_vers_dec("0") == 0
assert bin_vers_dec("1") == 1
assert bin_vers_dec("1101") == 13
assert bin_vers_dec("10110101") == 181
assert bin_vers_dec("00001101") == 13      # zéros à gauche : sans effet
print("Tous les tests passent !")
```

4. Du décimal au binaire

On fait l'opération inverse : on divise successivement par 2. Chaque reste est un bit, et le premier reste obtenu est le bit de poids faible (celui de droite). On s'arrête quand le quotient vaut 0.

n	$q = \lfloor n/2 \rfloor$	$r = n \bmod 2$
156	78	0
78	39	0
39	19	1
19	9	1
9	4	1
4	2	0
2	1	0
1	0	1

on lit les restes de bas en haut

$(156)_{10} = (10011100)_2$

Côté code, le point délicat est l'ordre : le premier reste calculé doit finir tout à droite. On construit donc le résultat en ajoutant chaque nouveau bit à gauche de la chaîne (c'est l'astuce de l'exercice 2).

ASTUCE

Deux situations à traiter à part : $n = 0$ (la boucle ne s'exécute jamais et la chaîne reste vide, or la bonne réponse est `"0"`) et les nombres négatifs, ignorés ici.

Exercice 7 À la main ★

Convertis en binaire en posant les divisions successives sur ton cahier.

45 → _____ 64 → _____

200 → _____ 255 → _____

Exercice 8 Coder les divisions successives ★

Complète la fonction.

```
def dec_vers_bin(n):
    if n == _____:
        return "0"
    resultat = ""
    while _____________:
        reste = _____________
        n = _____________
        resultat = _____________________
    return resultat
```

Exercice 9 Comprendre l'ordre ★★

1) Que renverrait la fonction si on écrivait `resultat = resultat + str(reste)` ? Teste avec `dec_vers_bin(6)`.
2) Quelle fonction de la partie 2 permettrait de « réparer » ce résultat ?

Exercice 10 Sur 8 bits ★★

En informatique, on écrit souvent un octet avec exactement 8 bits (par exemple $(5)_{10}$ devient `00000101`). Complète la fonction qui complète par des zéros à gauche.

```
def dec_vers_bin_8(n):
    s = dec_vers_bin(n)
    while _______________:
        s = _____________
    return s
```

Exercice 11 Le test aller-retour ★★

Les deux fonctions doivent être inverses l'une de l'autre. Explique ce que fait ce test, puis exécute-le.

```
for n in range(256):
    assert bin_vers_dec(dec_vers_bin(n)) == n
print("Les 256 valeurs sont correctes !")
```

5. L'hexadécimal

En base 16, il faut seize chiffres : les dix chiffres habituels puis les lettres `A` à `F`.

Décimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hexa	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Binaire	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

La bonne nouvelle

Les algorithmes sont identiques à ceux du binaire, en remplaçant 2 par 16. La seule difficulté est de passer entre un caractère (`"B"`) et sa valeur (`11`). Une chaîne contenant tous les chiffres résout élégamment le problème : la position d'un caractère est sa valeur.

Exercice 12 À la main ★

Convertis (pense que $(3E8)_{16} = 3 \times 16^2 + 14 \times 16 + 8$).

`B5`	→	_____	255	→	_____
`1A`	→	_____	100	→	_____
`7C`	→	_____	171	→	_____
`3E8`	→	_____	4095	→	_____

Exercice 13 Passer du caractère à sa valeur (et inversement)

```
CHIFFRES = "0123456789ABCDEF"

def valeur_chiffre(c):          # "B" -> 11
    return CHIFFRES._____________

def chiffre_valeur(v):          # 11 -> "B"
    return CHIFFRES[_____]
```

Exercice 14 Hexadécimal vers décimal ★

Reprends la méthode de Horner avec la base 16.

```
def hex_vers_dec(s):
    s = s.upper()           # accepte aussi "b5"
    valeur = 0
    for c in s:
        valeur = valeur * _____ + ____________________
    return valeur
```

Exercice 15 Décimal vers hexadécimal ★

Reprends cette fois la structure de `dec_vers_bin`. Ici la division se fait par 16 et le reste est un nombre entre 0 et 15 : comment le transformer en chiffre ?

```
def dec_vers_hex(n):
    if n == 0:
        return "0"
    resultat = ""
    while n > 0:
        resultat = ____________________________ + resultat
        n = _____________
    return resultat
```

Exercice 16 Généraliser à n'importe quelle base ★★★

Les quatre fonctions que tu as écrites se ressemblent beaucoup. Écris-en deux versions génériques qui fonctionnent pour toute base $b \leq 16$.

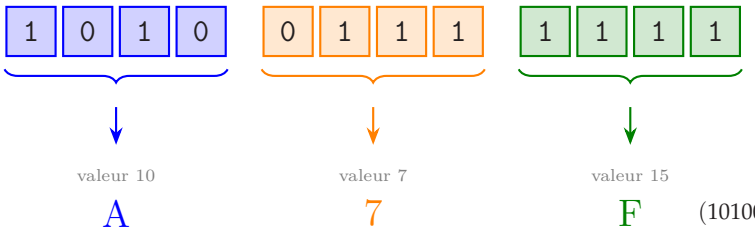
```
def vers_decimal(s, b):
    valeur = 0
    for c in s:
        valeur = valeur * _____ + valeur_chiffre(c)
    return valeur

def depuis_decimal(n, b):
    if n == 0:
        return "0"
    resultat = ""
    while n > 0:
        resultat = CHIFFRES[____________] + resultat
        n = _____________
    return resultat
```

Teste-les en base 2, 16, puis en base 8 et en base 3 : que valent `depuis_decimal(100, 8)` et `vers_decimal("2101", 3)` ? Réécris ensuite `bin_vers_dec(s)` en une seule ligne grâce à `vers_decimal`.

6. Passer directement du binaire à l'hexadécimal

Comme $16 = 2^4$, un chiffre hexadécimal correspond exactement à un paquet de 4 bits. Plus besoin de passer par le décimal : on découpe la chaîne binaire en paquets de 4 en partant de la droite, et on traduit chaque paquet avec la table.



$(101001111111)_2 = (A7F)_{16}$

Le sens inverse est tout aussi simple : chaque chiffre hexadécimal se transforme en exactement 4 bits.

PIÈGE

Dans le sens binaire → hexa, si le nombre de bits n'est pas un multiple de 4, il faut compléter à gauche avec des zéros avant de découper (`1111111` devient `0111 1111`, soit `7F`). Dans le sens hexa → binaire, il faut toujours 4 bits par chiffre : `7` s'écrit `0111`, pas `111`, sinon les paquets se « décalent ».

Exercice 17 À la main ★

`11010110` → _____ `3C` → _____

`1001111` → _____ `E9` → _____

Exercice 18 Du binaire à l'hexadécimal ★★

] Ici on lit la chaîne de gauche à droite : le résultat peut donc être construit normalement, en ajoutant à droite. Complète le code (tu as le droit de réutiliser `bin_vers_dec` et `CHIFFRES`).

```
def bin_vers_hex(s):
    # 1) compléter à gauche avec des 0 pour obtenir un multiple de 4
    while ____________________:
        s = "0" + s
    resultat = ""
    # 2) parcourir la chaîne par paquets de 4
    for i in range(0, len(s), _____):
        paquet = s[i:_________]
        resultat = resultat + CHIFFRES[_____________________]
    return resultat
```

Exercice 19 De l'hexadécimal au binaire ★★

```
def hex_vers_bin(s):
    resultat = ""
    for c in s:
        v = ____________________
        bits = ____________________
        while len(bits) < _____:
            bits = "0" + bits
        resultat = resultat + bits
    return resultat
```

Exercice 20 Pourquoi ça marche ? ★

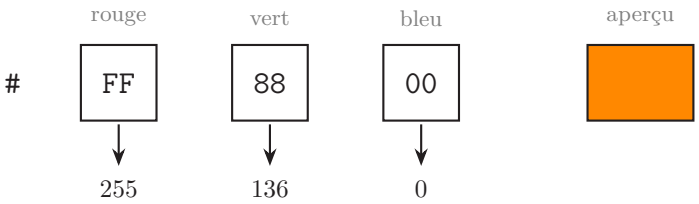
Explique avec tes mots (ou avec une égalité) pourquoi $(1011\,0101)_2$ vaut $11 \times 16 + 5$.

> **Indice**
>
> $(1011\,0000)_2 = (1011)_2 \times 2^4$.

7. Défis

Exercice 21 Couleurs sur le web ★★★

Une couleur `#FF8800` est composée de trois octets en hexadécimal : rouge, vert, bleu. Chaque canal est un entier entre 0 et 255.



1) Complète la fonction qui extrait les trois canaux.
2) Écris la fonction réciproque `rgb_vers_couleur(r, g, b)`. Attention : `dec_vers_hex(0)` renvoie `"0"` alors qu'il faut `"00"`.

```
def couleur_vers_rgb(code):              # code = "#FF8800"
    rouge = hex_vers_dec(code[1:_____])
    vert  = hex_vers_dec(code[_____:5])
```

```
    bleu  = hex_vers_dec(code[_____:_____])
    return (rouge, vert, bleu)          # -> (255, 136, 0)
```

Exercice 22 Additionner en binaire ★★★

On additionne comme à l'école primaire, colonne par colonne en partant de la droite, avec une retenue.

> **Exemple**
> $(1011)_2 + (110)_2 = (10001)_2$ (soit $11 + 6 = 17$).

retenues	1	1	1	0	
		1	0	1	1
+		0	1	1	0
	1	0	0	0	1

Cette figure est un indice, pas la solution : observe-la bien, puis complète le code.

```
def addition_binaire(a, b):
    n = max(len(a), len(b))
    while len(a) < n:
        a = "0" + a
    while len(b) < n:
        b = "0" + b
    retenue = 0
    resultat = ""
    for i in range(n - 1, ______, -1):
        somme = int(a[i]) + int(b[i]) + ___________
        resultat = str(___________) + resultat
        retenue = ___________
    if _________________:
        resultat = "1" + resultat
    return resultat
```

Exercice 23 Un convertisseur universel ★★★

Écris une fonction `convertir(s, b_source, b_dest)` qui convertit une chaîne d'une base quelconque vers une autre.

> **Indice**
> Le décimal sert de « pivot » entre deux fonctions de l'exercice 16. Écris ensuite un petit programme principal qui demande à l'utilisateur, avec `input()`, un nombre, sa base, puis la base souhaitée, et affiche le résultat.

Exercice 24 Pour aller plus loin (bonus) ★★★

1) Validité – écris `est_valide(s, b)` qui vérifie que tous les caractères de `s` sont des chiffres autorisés dans la base `b`.
2) Nombres négatifs – sur 8 bits, l'opposé de n s'écrit comme $256 - n$ (complément à deux). Écris `oppose_8bits(n)` : par exemple `oppose_8bits(5)` doit renvoyer `"11111011"`.
3) Nombres à virgule – comment écrire 0,625 en binaire ? (Multiplie par 2 plutôt que de diviser.)

8. Bilan

LES 6 ERREURS LES PLUS FRÉQUENTES

1) Oublier le cas $n = 0$ dans `dec_vers_bin` et `dec_vers_hex`.
2) Construire le résultat dans le mauvais sens (`resultat + reste` au lieu de `reste + resultat`) dans les conversions depuis le décimal.
3) Confondre le caractère `"7"` et l'entier `7` (pense à `int(c)`, `str(r)` ou `valeur_chiffre`).
4) Oublier de compléter les zéros à gauche (paquets de 4 bits, octets de 8 bits).
5) Utiliser `/` (flottant) au lieu de `//` (quotient entier).
6) Oublier `.upper()` et rejeter `"b5"` alors que c'est un nombre valide.

Batterie de tests finale

```
assert bin_vers_dec("10110101") == 181
assert dec_vers_bin(181)        == "10110101"
assert hex_vers_dec("B5")       == 181
assert dec_vers_hex(181)        == "B5"
assert bin_vers_hex("10110101") == "B5"
assert hex_vers_bin("B5")       == "10110101"
assert bin_vers_hex("1111111")  == "7F"
assert hex_vers_bin("7F")       == "01111111"
for n in range(1000):
    assert bin_vers_dec(dec_vers_bin(n)) == n
    assert hex_vers_dec(dec_vers_hex(n)) == n
    b = dec_vers_bin(n)
    assert bin_vers_dec(hex_vers_bin(bin_vers_hex(b))) == n
print("Bravo, tout fonctionne !")
```